

Scaling Strategy - Cheat Sheet

Scaling is the engineering of “what happens when traffic grows 10x?” The answer is rarely “just add servers”. Real scaling forces you to think about state, hotspots, dependencies, and the systems behind your systems.

There’s no single strategy. There’s a hierarchy of techniques, ordered roughly from cheap-to-apply to expensive-to-apply. The art is picking the right level for your actual bottleneck. The trendiest pattern isn’t always the right one.

Scale up vs scale out

Two fundamental directions.

Vertical scaling (scale up)

Make one machine bigger. More CPU, more RAM, faster disk.

Pros:

- No code changes needed.
- Simpler operationally.
- Works for stateful systems (databases) where horizontal scaling is hard.

Cons:

- Hard ceiling on the biggest machine you can buy.
- Single point of failure.
- Larger machines are disproportionately expensive.

Horizontal scaling (scale out)

Add more machines. Spread load across them.

Pros:

- Effectively unbounded. Add more nodes as needed.
- Better fault tolerance. Lose one node, others keep serving.
- Cheaper per unit of capacity at scale.

Cons:

- Requires the code to be stateless or carefully handle distributed state.
- Adds coordination, load balancing, and discovery problems.
- Distributed systems are harder to reason about.

For modern systems, horizontal is the default. Vertical scaling is the fallback for stateful pieces (databases especially).

The scaling hierarchy

When traffic grows, work through these in order. Earlier levels are cheaper and less invasive.

1. Improve the slow path. Profile first. A bad query or N+1 problem can look like a scaling issue.
2. Cache. A 90% hit rate reduces backend load by 10x. (See [caching-strategy.pdf](#).)
3. Add read replicas. Spread read traffic across multiple DB copies.
4. Scale the application tier horizontally. Add more instances behind a load balancer.
5. Make work async. Push slow operations to a queue, return fast.
6. Use a CDN. Push static and cacheable content to the edge.
7. Shard the database. Split data across multiple DBs by key.
8. Decompose the service. Break the monolith. Scale hot paths independently.
9. Multi-region. Spread the system across geographic regions.

Most teams should stay in the top half for years. Sharding and multi-region are heavy machinery. Use them when you actually need them.

Stateless vs stateful

Stateless services scale trivially. Stateful ones don't.

Stateless

Each request is independent. The server holds no per-client state between requests.

```
Request -> Any instance -> Response
```

To scale, add more instances behind a load balancer.

Make stateless by pushing state to:

- The client (cookies, tokens).
- A shared cache (Redis).
- A database.

Most modern web services are designed stateless from day one.

Stateful

The server remembers things between requests (user session in memory, in-progress work, an active WebSocket).

To scale, you have to handle the state carefully:

- Sticky sessions: route the same user to the same instance. Easy, but breaks when an instance dies.
- Externalize the state: move it to Redis or a DB. Now the service is effectively stateless again.

- Partition the state: each instance owns a slice of the keyspace (consistent hashing, Kafka partitions).

WebSockets, long-running computations, and stateful caches are the awkward cases. They need explicit design choices. Adding more instances doesn't solve them on its own.

Load balancing

A load balancer distributes incoming traffic across multiple backend instances.

Layers

Layer	What it sees	Examples
L4 (transport)	TCP/UDP connections, no understanding of HTTP	AWS NLB, HAProxy in TCP mode
L7 (application)	HTTP requests, headers, paths	nginx, Envoy, AWS ALB, GCP Load Balancer

L7 is more flexible (route by path, header, cookie). L4 is faster and works for non-HTTP traffic.

Algorithms

Algorithm	Behavior
Round robin	Rotate through backends in order
Least connections	Send to the backend with the fewest active connections
Least response time	Send to the fastest backend
IP hash	Same client always hits the same backend (sticky)
Consistent hashing	Distributes load evenly, minimizes reshuffling when backends change
Random with two choices	Pick two random backends, send to the less-loaded. Surprisingly effective.

Least-connections and least-response-time adapt to real load. Round robin assumes equal capacity and equal request cost.

Health checks

Load balancers route only to healthy backends. Configure them carefully:

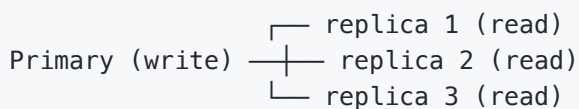
- Cheap. A health check should respond fast and not do real work.
- Honest. Return unhealthy when the instance can't actually serve traffic.
- Decoupled from heavy dependencies. A slow DB shouldn't fail the health check. That just shifts load to even slower instances.

Database scaling

Most scaling problems eventually trace back to the database. The DB is usually the hardest tier to scale.

Read replicas

Most workloads are read-heavy. Spin up replicas that follow the primary and serve reads.



Pros:

- Cheap to add.
- Scales reads near-linearly.

Cons:

- Replication lag. Replicas are slightly behind. Reads can be stale.
- Doesn't help with write throughput.

Read-your-writes can break. You write to the primary, then read from a replica that hasn't caught up yet. Pin recent writes to the primary or use synchronous replication for critical paths.

Vertical partitioning

Split the schema by feature. One DB for users, one for orders, one for inventory.

Pros:

- Each team owns its data. Aligns with service boundaries.
- Smaller DBs are easier to manage.

Cons:

- Joins across DBs disappear. Replace with API calls or denormalization.
- Distributed transactions for cross-domain writes (see [saga-pattern.pdf](#)).

Sharding (horizontal partitioning)

Split a single table across multiple DB instances by a shard key.

```
Users 0-999      -> Shard A
Users 1000-1999 -> Shard B
Users 2000-2999 -> Shard C
```

Or by hash of the user ID, by tenant, by region.

Pros:

- Scales writes (the hard one).
- Scales reads.
- Bounded data size per shard.

Cons:

- Cross-shard queries are slow or impossible.
- Resharding is painful.
- Pick the wrong shard key and you'll be stuck with it for years.

Common shard keys: user ID (per-user isolation), tenant ID (multi-tenant SaaS), geographic region.

Connection pooling

A common scaling killer is connection blow-up. Each app instance opens N connections, multiply by M instances, exceed the DB's connection limit, everything falls over.

Use a connection pool per instance (HikariCP for Java, PgBouncer in front of Postgres). Tune the pool size to balance: enough to handle peak, not so many that the DB chokes.

```
total_connections = instances * pool_size_per_instance
```

Keep `total_connections` under the DB's `max_connections`, with headroom for admin queries.

Async and queues

When something is slow, don't make the user wait. Acknowledge fast, process later.

```
HTTP request -> validate -> enqueue -> 202 Accepted
                        ↓
                    worker pool -> actual work
```

Good candidates for async:

- Sending email or push notifications.
- Image processing, video encoding.
- Report generation.
- Webhooks to third parties (which might be slow or flaky).
- Anything that takes longer than the user wants to wait.

Common tools:

Tool	Best for
Kafka	High-throughput event streams, ordering per partition
RabbitMQ	Traditional task queues, complex routing
AWS SQS	Managed, simple, good defaults
Google Pub/Sub	Managed, global
Redis Streams / Lists	Lightweight queues when you already use Redis
Temporal / Cadence	Long-running workflows with state and retries

Async decouples producers from consumers. Spikes get absorbed by the queue. Workers scale independently of the API tier.

CDN and edge caching

Push content close to the user.

What goes at the edge:

- Static assets (JS, CSS, images, fonts).
- Public API responses (catalog, search, anything not user-specific).
- Even dynamic pages, with smart cache rules.

Modern CDNs (CloudFront, Cloudflare, Fastly) also let you run code at the edge (Cloudflare Workers, Lambda@Edge, Fastly Compute). Useful for auth, redirects, A/B testing without round-tripping to the origin.

A cache hit at the edge means the request never touches your infrastructure. The biggest scaling win available for content that fits.

Auto-scaling

Have the platform add or remove capacity based on load.

Horizontal Pod Autoscaler (HPA)

Adds and removes pods based on CPU, memory, or custom metrics. Standard in Kubernetes. (See [kubernetes.pdf](#) for the full picture.)

```
spec:
  minReplicas: 3
  maxReplicas: 50
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70
```

Cluster Autoscaler

Adds and removes nodes when pods can't be scheduled or capacity is idle. A separate layer from HPA.

Vertical Pod Autoscaler (VPA)

Adjusts per-pod CPU and memory limits. Less common, conflicts with HPA.

Predictive autoscaling

Scale ahead of expected load. Useful when scale-up takes minutes and you have predictable traffic patterns (Monday morning, batch jobs, scheduled marketing pushes).

Scaling lag

Auto-scaling reacts to load. By the time it kicks in, you've already had degraded latency. Defenses:

- Conservative target utilization. Scale at 50-70% CPU. 90% leaves no margin for spikes.
- Pre-warm before known events. Marketing pushes, holiday traffic.
- Min replicas above zero. Cold starts kill latency.

Connection pools and limits

Scaling discovers limits you didn't know existed. Common ones:

- DB connections. Limited per instance, exhausted under load.
- File descriptors. OS-level limit per process.
- Thread pools. Block on a slow upstream, threads pile up, requests queue, latency spikes.
- Ephemeral ports. A client making many short-lived connections runs out of source ports.
- Memory. A pod with no limit grows until it OOMs.
- Concurrent requests per upstream. Hammering a third-party API hits their rate limit and your calls start failing.

Set explicit limits everywhere. Surface them as metrics. Alert before they max out.

Capacity planning

Two questions:

1. How much load do we serve today?
2. How much can a single instance handle?

$$\text{Capacity} = (\text{max per instance}) * (\text{number of instances}) * \text{safety factor}$$

A safety factor of 0.5-0.7 leaves room for spikes and partial failures. Running at 95% utilization means a single failure tips you over.

Run load tests. Find the breaking point per instance. Plan around 50% of that as the steady-state target.

Track these metrics:

- Requests per second (RPS). Per service, per endpoint.
- Latency percentiles. p50, p95, p99.
- Resource saturation. CPU, memory, IO, network.
- Queue depths. Message brokers, thread pools, request queues.

Capacity planning is just paying attention to these over time.

Identifying bottlenecks

Before scaling, find what's actually slow.

Bottleneck	Symptoms
CPU	Service maxes out CPU under load. Threads waiting on each other.
Memory	OOMs, GC pauses, swapping.
DB	DB CPU high, slow queries in logs, connection pool exhaustion.
Network	Packet loss, retransmits, throughput plateau.
External API	Latency spikes correlated with the upstream service.
Lock contention	Threads blocked on synchronized or DB row locks.
Cold cache	First request slow after deploy or scale-up.

Tools:

- APM (Datadog, New Relic, Honeycomb): traces and aggregations.
- Profilers (async-profiler, perf, pypsy): CPU and lock profiles.
- Database EXPLAIN. The single most useful tool for query performance.
- Distributed tracing (OpenTelemetry, Jaeger): see where time goes across services.

The most common mistake is scaling the wrong tier. If the DB is the bottleneck, doubling the application instances just doubles the load on the DB.

Common anti-patterns

- Premature sharding. Adds complexity for years before it pays off. Vertical scaling and read replicas usually buy you enough time.
- Microservices for scale. Splitting a service doesn't make the slow path faster. Often makes it slower. Fix the slow path first.
- Distributed monolith. Many services, all coupled through synchronous calls. Worst of both worlds. Decouple with async or rejoin.
- Auto-scaling without limits. Pods scale up infinitely until the DB falls over. Always set maxReplicas.
- Caching as a band-aid. Caching a query that should be fixed hides the real problem and grows complexity.
- No backpressure. Producers send faster than consumers can process. Queues grow unboundedly until memory dies.
- Scaling the front when the back is the problem. More frontend pods making more DB calls makes the DB slower.
- Single-region everything for a global product. Latency to far-away users will dominate. Use a CDN or go multi-region.

Best practices

- Measure first. Profile, trace, look at metrics. Don't guess.
- Cache before you shard. Cheaper, less invasive.
- Make new services stateless by default. Save state complications for when they're necessary.
- Use async for slow work. Don't make users wait on something that doesn't need to be synchronous.
- Set limits everywhere. Connection pools, thread pools, queue sizes, retries, timeouts.
- Plan for partial failures. A region going down should leave the system degraded but still functional.
- Load test before launch. Knowing your breaking point in dev beats finding it in prod.
- Pre-warm before known spikes. Don't trust auto-scaling alone for predictable traffic.
- Right-size resource requests. Over-allocated pods waste cluster capacity. Under-allocated pods get OOM-killed.

- Plan for the next 10x. Designing for 1000x wastes effort on problems you don't have yet. Target the next horizon, then iterate.

Common gotchas

- Database connections multiply. 100 pods * 50 connections each = 5,000 connections to a DB that allows 200. Use a pooler like PgBouncer.
- Thread pool starvation. A slow upstream blocks all threads. Use bulkheads or async I/O. (See [concurrency-resilience.pdf](#).)
- Hot keys. One row, one cache key, one user gets disproportionate traffic. Shard it, replicate it, or rate limit it.
- Replication lag. Reading from a replica right after writing to the primary returns stale data. Pin recent writes to the primary.
- Resharding pain. Picking a shard key is a one-way decision. Get it wrong and you'll spend months migrating.
- Cache stampede on cold start. Restart drops the cache. Hundreds of misses hit the DB. Pre-warm or use single-flight.
- Auto-scaling lag. New instances take 30-90s to come up. Set min replicas to handle the bottom of your traffic curve plus margin.
- Geographic latency. A user in Sydney hitting your us-east-1 service eats 200ms of round-trip latency on every call. CDNs and regional deploys help.
- Per-tenant noisy neighbors. One big tenant uses 80% of your capacity. Rate limit per tenant. Shard by tenant.
- Background jobs scheduled at the same time. Every node fires its cron job at the top of the hour. Adds a giant load spike. Stagger them.
- Forgetting the dependencies. Your service scales fine. The Redis cluster behind it doesn't. The third-party SMS API rate limits you. Capacity plan the whole chain.

Quick reference

Concept	Key fact
Vertical scaling	Bigger machine. Simple, has a ceiling.
Horizontal scaling	More machines. Default for stateless services.
Stateless service	Scales by adding instances behind a load balancer
Read replicas	Scale reads. Watch for replication lag.
Sharding	Scale writes. Picking the shard key is a long-term decision.
Connection pooling	Bound DB connections per instance.
Async work	Decouple producer from consumer with a queue.
CDN	Highest-impact scaling for static and cacheable content.
HPA	Auto-scale pods based on CPU or custom metrics.
Cold start	New instances take time. Set sensible min replicas.
Bottleneck	Always find the real one before scaling.
Safety factor	Run at 50-70% capacity to absorb spikes.